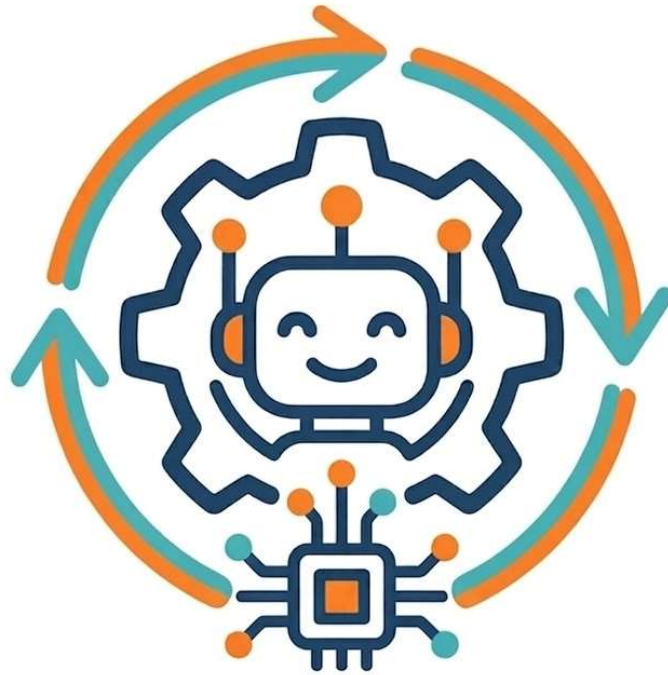


UADER

FCyT



CONTROL AUTOMÁTICO

PRACTICA EDUCATIVA TERRITORIAL
FACULTAD DE CIENCIA Y TECNOLOGÍA - FCyT
UADER FCyT

Índice

	Pág
Introducción:	1
Marco Teórico:	1
Partes del proyecto:	2
Documentación de lo programado:	11
Clases utilizadas	11
Descripción breve de cada clase:	13
Conceptos de Programación Orientada a Objetos aplicados:	13
Programa principal:	14
Flujo del programa principal:	17
Servidor Web/programación Web:	21
Construcción física:	23
Detalles Adicionales:	24
Bibliografía:	25



Introducción:

Este trabajo tiene por finalidad desarrollar un sistema IoT (de internet de las cosas) de un control abarcando los contenidos de la cátedra. Los temas a abarcar/aplicar son los siguientes:

Programación Orientada a Objetos.

Protocolos de comunicaciones digitales.

Desarrollo web.

Gestión de versiones (Git).

Despliegue básico en la nube.

El proyecto escogido fue un calentador de agua, un control PID, con fines pedagógicos. La idea es que, en última instancia, un docente o un alumno pueda controlar desde su teléfono o computadora las variables del control PID.

Marco Teórico:

Es conveniente dar una pequeña aclaración de los conceptos a utilizar:

Programación Orientada a Objetos (POO):

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el código en torno a "objetos", entidades que combinan datos (atributos) y comportamientos (métodos). Este enfoque busca modelar problemas del mundo real de manera más natural, facilitando la reutilización, escalabilidad y mantenibilidad del software. Haciendo una analogía, es como si la programación estructurada fuera como la aritmética, donde se siguen pasos para, por ejemplo, dividir dos números; mientras que la programación orientada a objetos es más parecida a la mecánica, tenemos objetos que hacen cosas, con características y que pertenecen a clases distintas. La POO tiene cuatro pilares fundamentales: [abstracción](#),

[encapsulamiento](#), [herencia](#) y [polimorfismo](#). *Encapsulación:*

Mecanismo que restringe el acceso directo a los datos internos de un objeto, exponiendo solo una interfaz controlada que garantiza la integridad del estado. *Herencia:*

Relación jerárquica que permite a una subclase derivar atributos y métodos de una superclase, facilitando la reutilización y extensión del código. *Polimorfismo:*

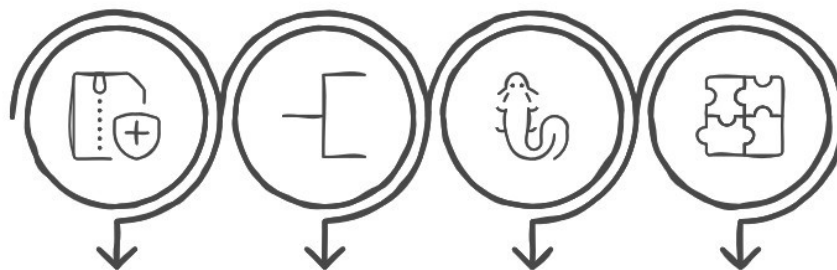
Capacidad de un objeto para adoptar múltiples formas, permitiendo que una misma interfaz se comporte de manera distinta según la implementación concreta en tiempo de ejecución.

Abstracción:

Principio que consiste en representar las características esenciales de un objeto, ocultando los detalles de implementación mediante interfaces o clases abstractas. *Composición:*

Técnica de diseño donde un objeto contiene instancias de otros objetos como partes, delegando responsabilidades y estableciendo relaciones de pertenencia. Dicho y visto de otra forma, una clase que se compone de otras clases

Pilares de la programación orientada a objetos



Encapsulación

Protege los datos restringiendo el acceso.

Herencia

Permite que las clases hereden propiedades y métodos.

Polimorfismo

Permite que los objetos tomen muchas formas.

Abstracción

Oculto detalles de implementación complejos.

Made with Napkin

Pangolín:

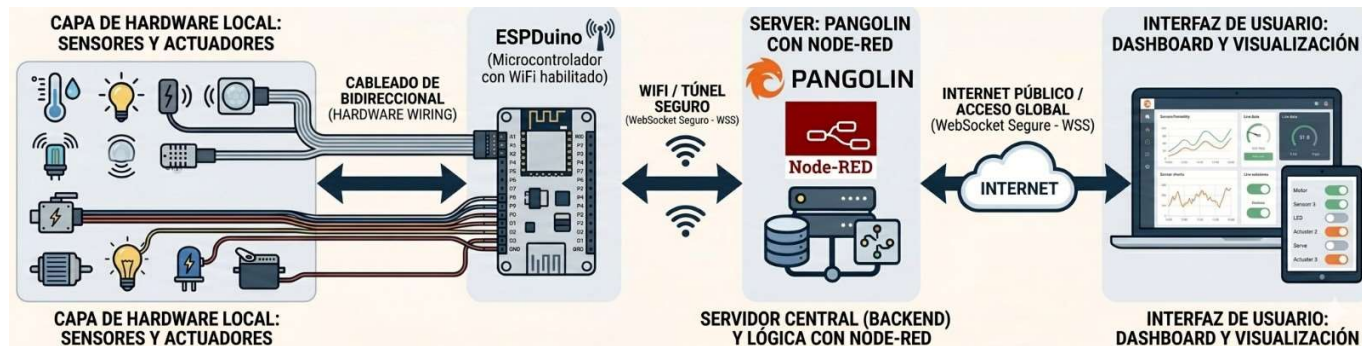
Pangolin es una plataforma de código abierto que, mediante una arquitectura cliente-servidor sobre WireGuard, integra proxy inverso y VPN para exponer recursos de red de forma segura sin requerir IPs públicas ni puertos abiertos. Se estructura en un servidor central con panel de administración, sites (conectores ligeros con túneles salientes permanentes) y clientes para los usuarios finales. El flujo despliega el servidor, configura los sites conectados vía túneles cifrados y define recursos públicos o privados con políticas de mínimo privilegio. El servidor autentica al usuario —por navegador o mediante el cliente para servicios como SSH/RDP— y enruta el tráfico al site correspondiente, centralizando así la autenticación, autorización y auditoría. Todo el tráfico se cifra con WireGuard, eliminando la necesidad de configurar NAT o abrir puertos en los routers locales. Es lo que utilizamos para exponer la aplicación de nuestro proyecto sin pagar hosting ni abrir puertos o similares.

Node-red:

Node-RED es una herramienta de programación visual de código abierto basada en Node.js (Javascript), diseñada originalmente por IBM para conectar hardware, APIs y servicios en línea. Su interfaz basada en navegador permite construir aplicaciones conectando nodos en flujos, lo que simplifica la creación de lógica compleja sin necesidad de codificación extensa. En el ámbito del IoT, Node-RED sobresale como un "pegamento" que integra dispositivos, sensores, protocolos como MQTT y HTTP, y servicios en la nube. Su arquitectura modular y su amplio repositorio de nodos comunitarios facilitan la agregación de datos, la traducción de protocolos, la automatización de reglas y la visualización de información. En nuestro proyecto hace de Backend-Frontend.

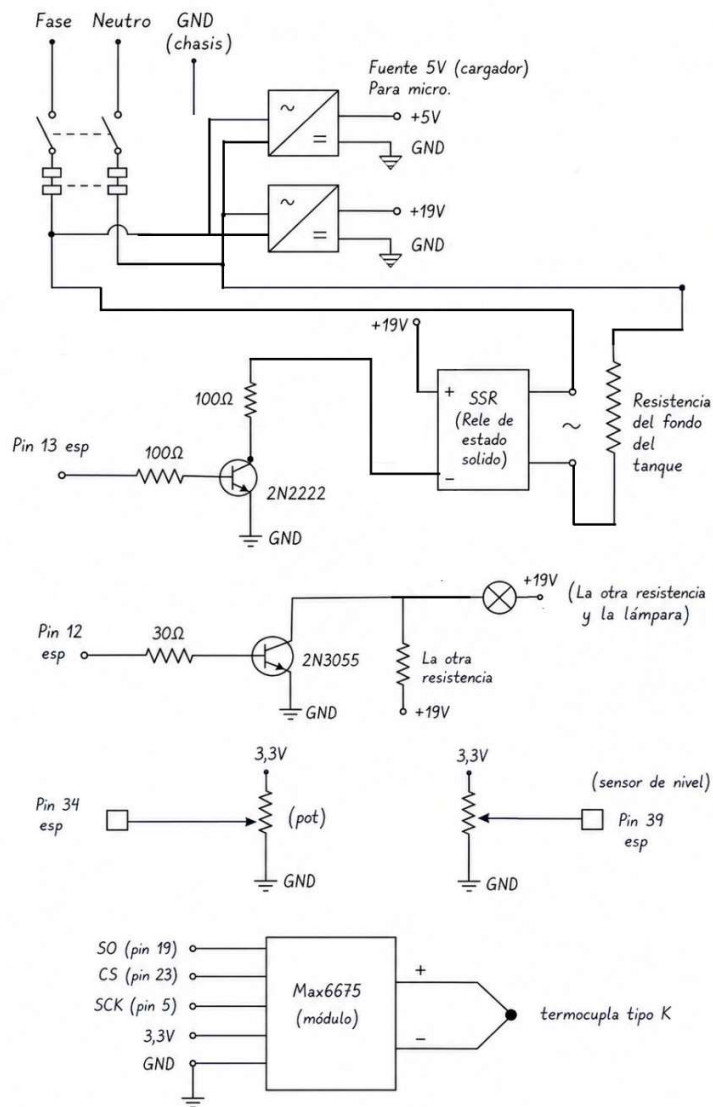
Partes del proyecto:

Lo siguiente es una breve aclaración del proyecto y como lo hemos desarrollado y desplegado. Podemos inicialmente separar lo desarrollado en 4 grandes secciones con comunicación bidireccional, como se ve a continuación:



1. Hardware local:

Son los distintos actuadores y sensores que interactúan con el microcontrolador. Podríamos definirlo con el siguiente esquemático:



Se compone esencialmente de dos actuadores y tres sensores:

- Dos actuadores: Una resistencia tipo herradura de 66 Ω sometida a 220 v mediante un relé de estado sólido (con un transistor 2n222 como driver) al que llamaremos de ahora en adelante “resistencia relé”, y otra resistencia igual con una lámpara, pero sometida a 19 v comandado por un transistor 2n3055.

- Dos sensores: Uno que es simplemente un potenciómetro en modo de divisor resistivo (esto para poder “simular un error”, esto permite explicar conceptos y de sintonía a lazo abierto fácilmente), el otro es el módulo MAX6675. El MAX6675 es un circuito integrado diseñado para medir temperatura mediante una termocupla tipo K, realizando la conversión de la señal analógica a datos digitales listos para ser procesados por un microcontrolador. Incorpora un convertidor analógico-digital (ADC) de 12 bits y compensación automática de unión fría, lo que simplifica significativamente el sistema de medición de temperatura. Recibe la pequeña diferencia de voltaje generada por la termocupla tipo K y la acondiciona internamente para su conversión digital. su tiempo de conversión de señal típica es de 220ms.



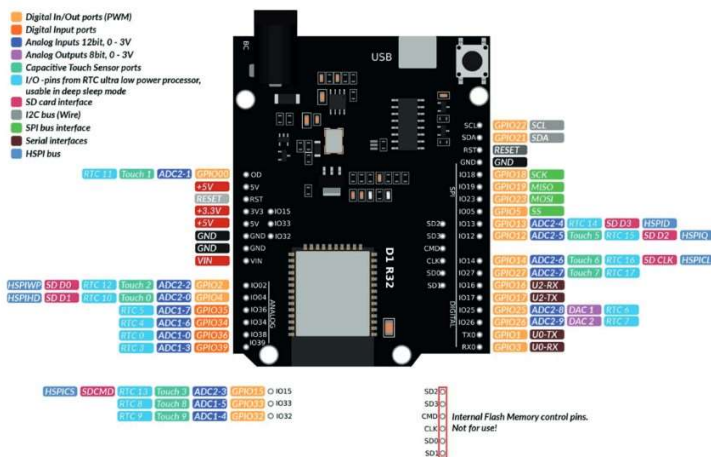
Lo escogimos porque es relativamente simple, con amplio rango y sumergible.

- Un sensor de nivel que funciona básicamente como un potenciómetro, donde el cursor es el agua, que se lee con una lectura analógica.

2. Espduino (microcontrolador):

El microcontrolador escogido para el proyecto fue una presentación del esp32 concretamente la placa D1 R32, comercialmente conocido “EspDuino”:

D1 R32 Board
Pinout



El **ESPDUINO-32** es una placa de desarrollo basada en el módulo **ESP-32** de Espressif Systems, fabricada por la compañía Shenzhen Doit Technology Co., Ltd.. Integra conectividad **WiFi**, **Bluetooth 4.2**, **Ethernet** y capacidad para transmisión de video en tiempo real, y es compatible con todas las placas de expansión de Arduino. Básicamente eso, un esp32 con presentación de Arduino.

Especificaciones técnicas principales

Categoría	Parámetro
CPU	Xtensa LX6 de 32 bits (single/dual core), hasta 600 DMIPS ¹
Memoria	448 KB ROM, 520 KB SRAM, 16 KB SRAM en RTC ¹
Almacenamiento	Hasta 4 memorias Flash/SRAM QSPI, máx. 16 MB cada una ¹
Voltaje de alimentación	2.2V – 3.6V ¹
Alimentación (placa)	5V – 12V (por USB o DC) ¹
Consumo medio	80 mA ¹
Dimensiones	66 mm × 53 mm, altura 13.5 mm ¹
Temperatura de trabajo	-40°C a 85°C ¹
Interfaces	12-bit ADC (18 canales), 2× DAC, 10× sensores táctiles, 4× SPI, 2× I2S, 2× I2C, 3× UART, SDIO, Ethernet MAC con DMA, CAN 2.0, PWM, IR, etc. ¹

3. Servidor tunelizado (Pangolin + node red);

Lo primero que hicimos fue configurar el pc a usar como servidor como sitio para node-red con un recurso asignado:



Administrar Sitios

Crear y administrar sitios para permitir la conectividad a redes privadas

Conectar cualquier red

Un sitio es una conexión a una red remota que permite a Pangolin proporcionar acceso a recursos, públicos o privados, a usuarios en cualquier lugar. Instale el conector de red del sitio (Newt) en cualquier lugar donde pueda ejecutar un binario o contenedor para establecer la conexión.

Q

Buscar sitios...

Nombre	Estado	Tiempo de actividad (30d)	Datos en	Datos Fuentes	Tipo	Recursos
aaaaad	● Desconectado	4.8%	47.85 MB	298.42 MB	Newt v1.8.1 ⓘ	2 Recursos ▾

Sabemos que el nombre “aaaaad” es poco intuitivo, pero era el sitio que se había configurado para un proyecto anterior. Este sitio estará “apuntando” al puerto 1880 de nuestra pc, como la pc es siteconnector y recurso a la vez, este es “local host” es decir 127.0.0.1.

Hecho eso, le asignamos un recurso, este será nuestro dominio del proyecto:



Administrar recursos públicos

Crear y administrar recursos que sean accesibles públicamente a través de un navegador web

Acceso Público basado en Web

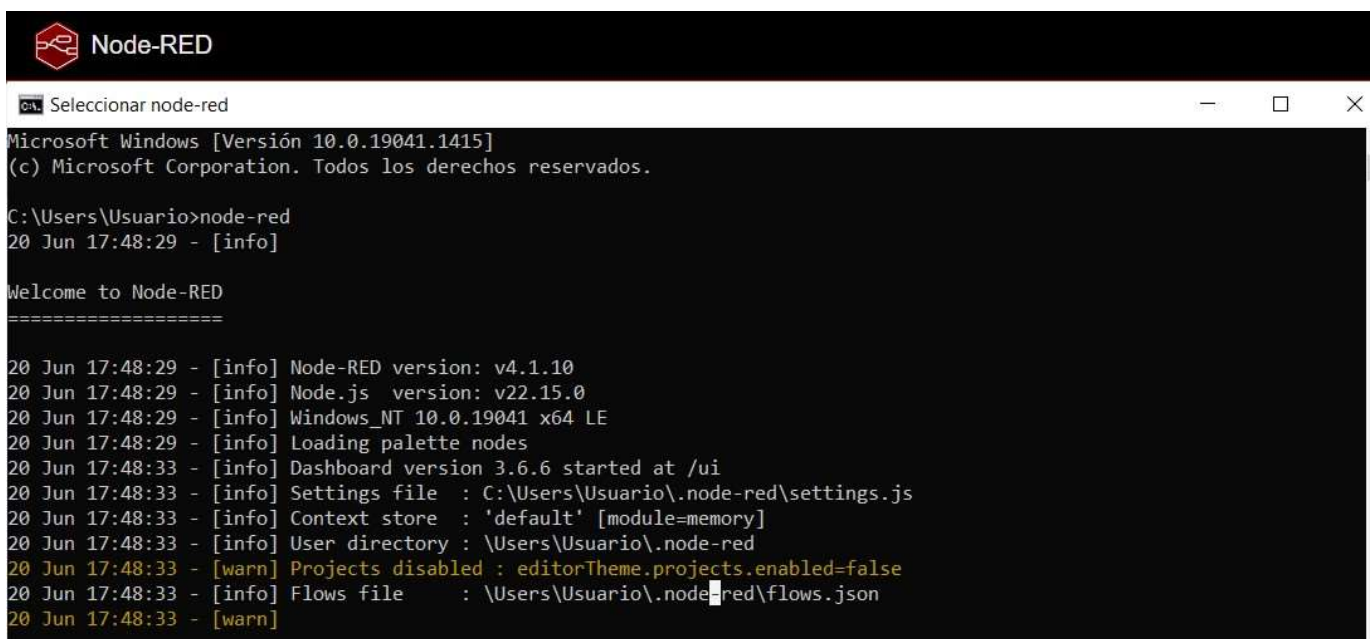
Los recursos públicos son proxies HTTPS accesibles para cualquiera en Internet a través de un navegador web. A diferencia de los recursos privados, no requieren software del lado del cliente e incluyen políticas de acceso basadas en identidad y contexto.

TrabajoFinalUADER • 1 sitio • ⌚ Desconocido • Sin datos • <https://trabajofinalintegradoruader.hostlocal.app> No protegido • Editar →

Siendo pues finalmente nuestro dominio:

<https://trabajofinalintegradoruader.hostlocal.app/ui/>

(/ui es para que abra el dashboard de node red y no la vista de desarrollo). Ejecutamos newt (para exponer el servicio) y node-red:



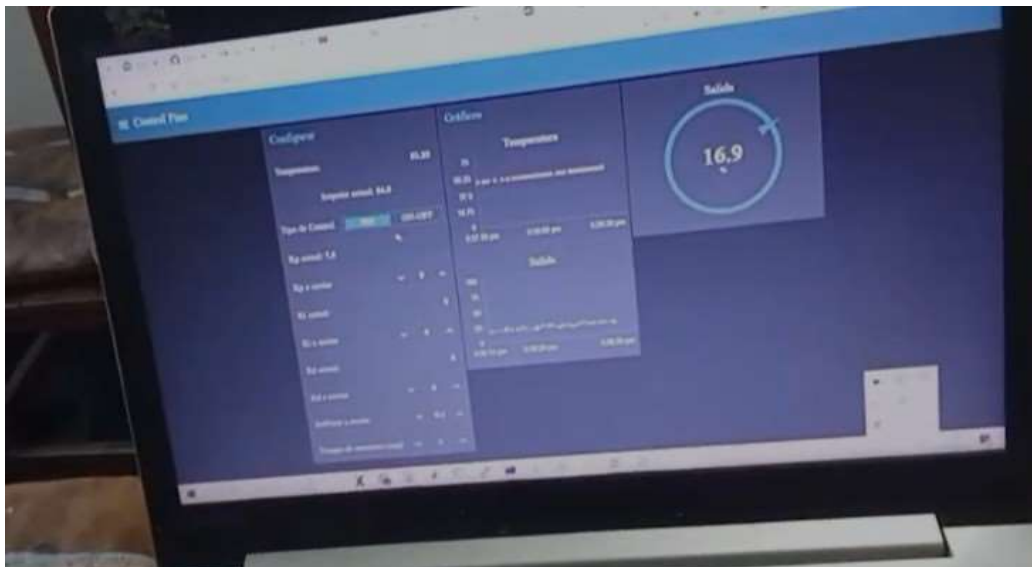
```
Node-RED
Selecionar node-red
Microsoft Windows [Versión 10.0.19041.1415]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario>node-red
20 Jun 17:48:29 - [info]

Welcome to Node-RED
=====

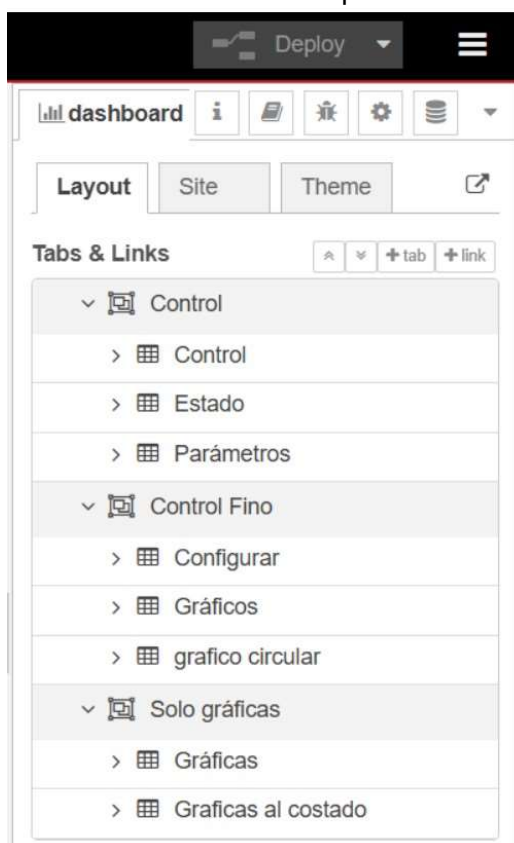
20 Jun 17:48:29 - [info] Node-RED version: v4.1.10
20 Jun 17:48:29 - [info] Node.js version: v22.15.0
20 Jun 17:48:29 - [info] Windows_NT 10.0.19041 x64 LE
20 Jun 17:48:29 - [info] Loading palette nodes
20 Jun 17:48:33 - [info] Dashboard version 3.6.6 started at /ui
20 Jun 17:48:33 - [info] Settings file : C:\Users\Usuario\.node-red\settings.js
20 Jun 17:48:33 - [info] Context store : 'default' [module=memory]
20 Jun 17:48:33 - [info] User directory : \Users\Usuario\.node-red
20 Jun 17:48:33 - [warn] Projects disabled : editorTheme.projects.enabled=false
20 Jun 17:48:33 - [info] Flows file : \Users\Usuario\.node-red\flows.json
20 Jun 17:48:33 - [warn]
```

4. Interfaz de usuario (parte visual
“ux/ui”) Esto es sencillamente los

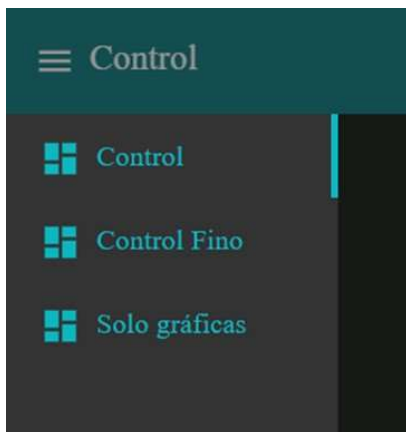


2 Punto de vista de un usuario de pc

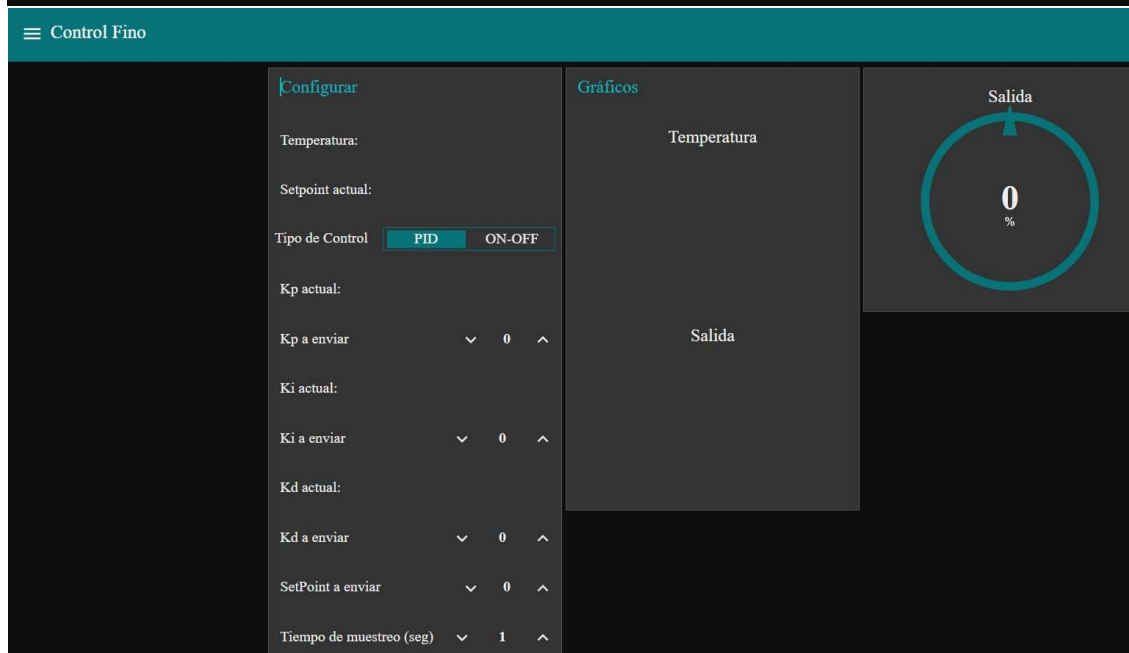
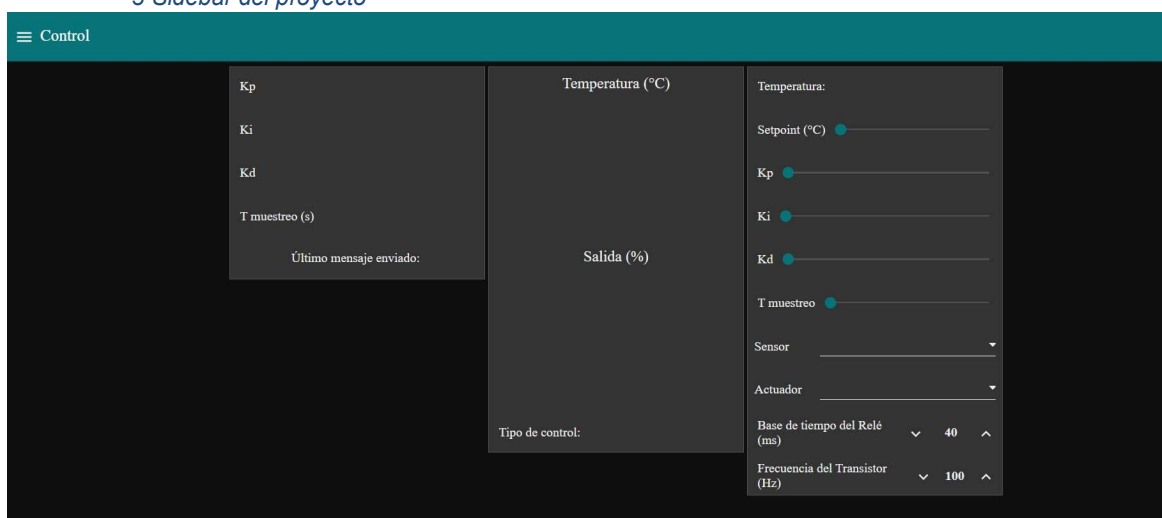
Lo realizado fue realizar varias pestañas desplegables con grupo donde se despliegan los distintos componentes de ui del dashboard por defecto de node-red:

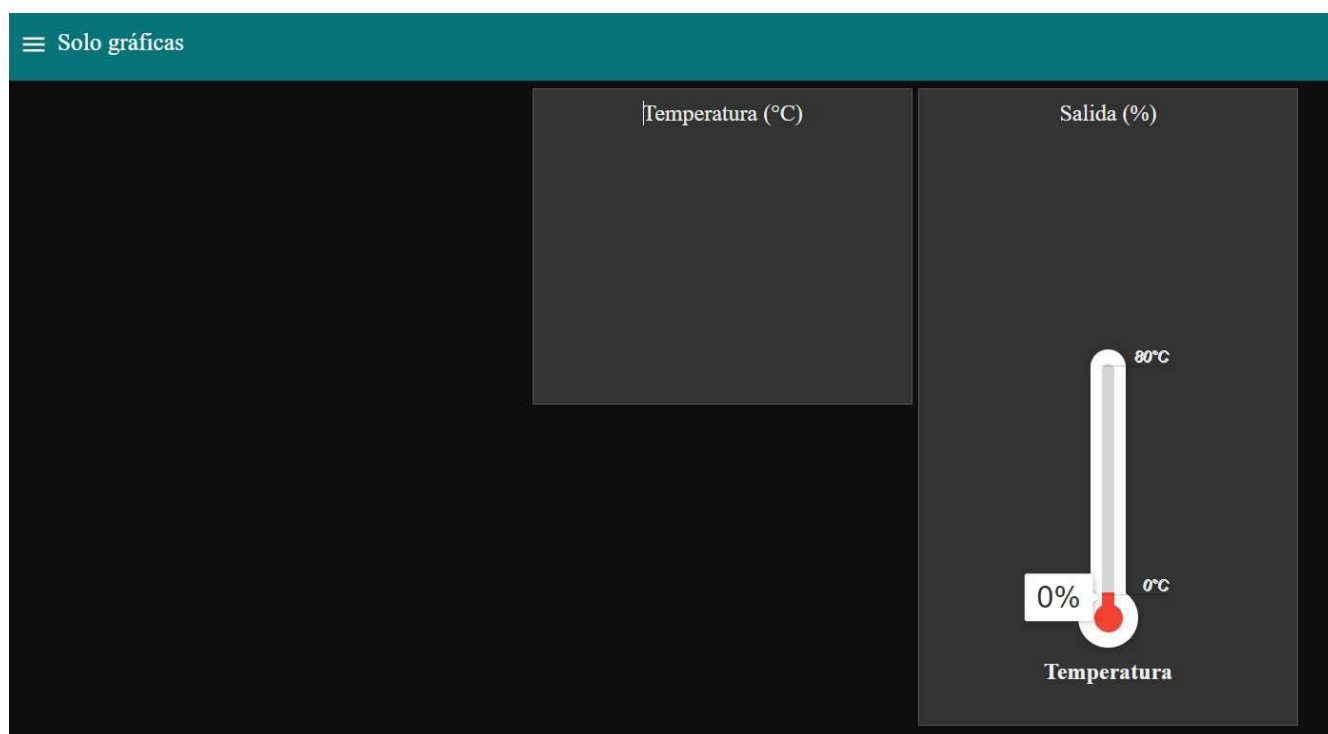


El Dashboard por defecto es sencillamente distintos nodos que mezclan css y funciones en javascript que corren del lado del usuario; esto hace veloz el servidor, lo que permite que varios estudiantes puedan ver al mismo tiempo sin saturarse. Veamos un detalle de cada vista (sin datos cargados por el momento):

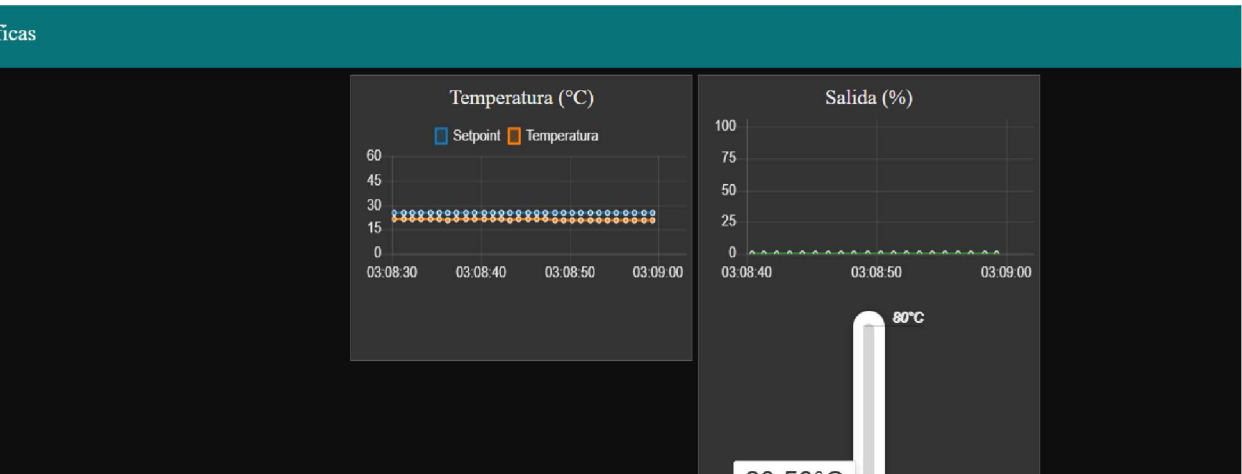
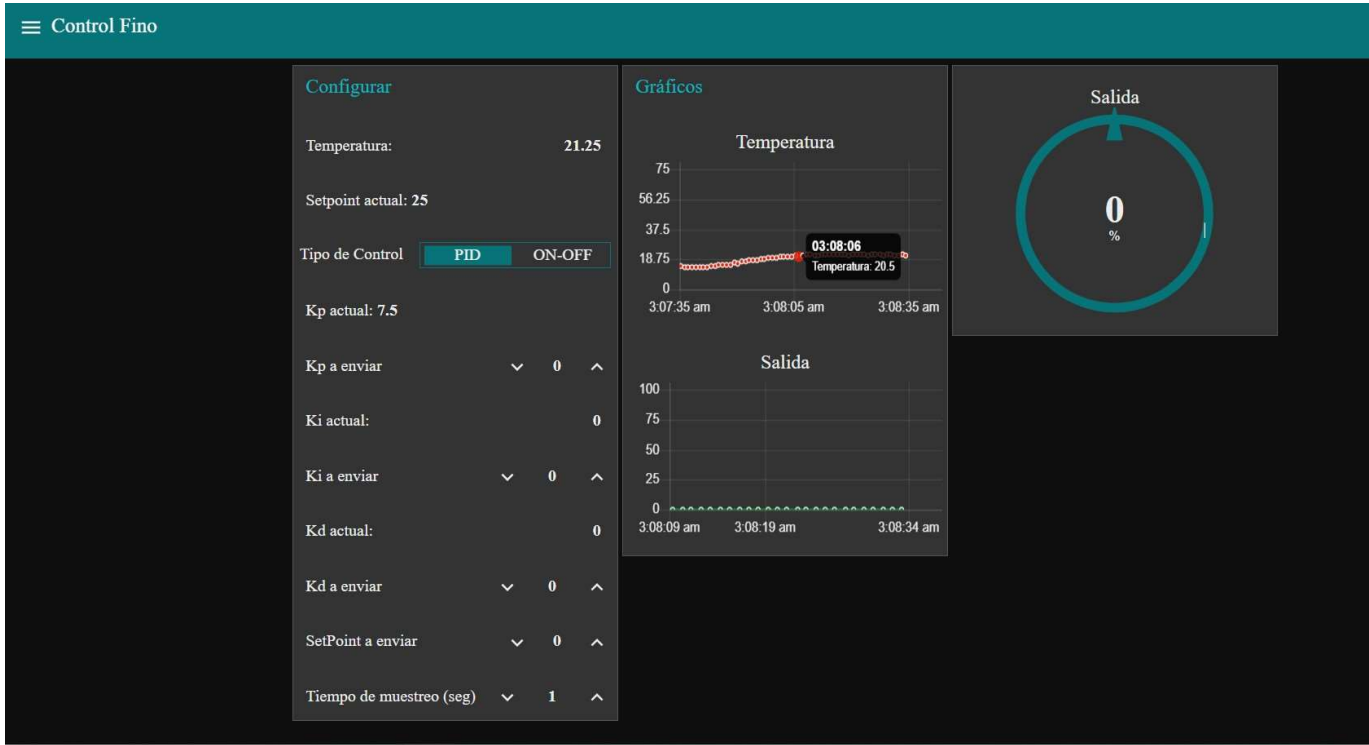
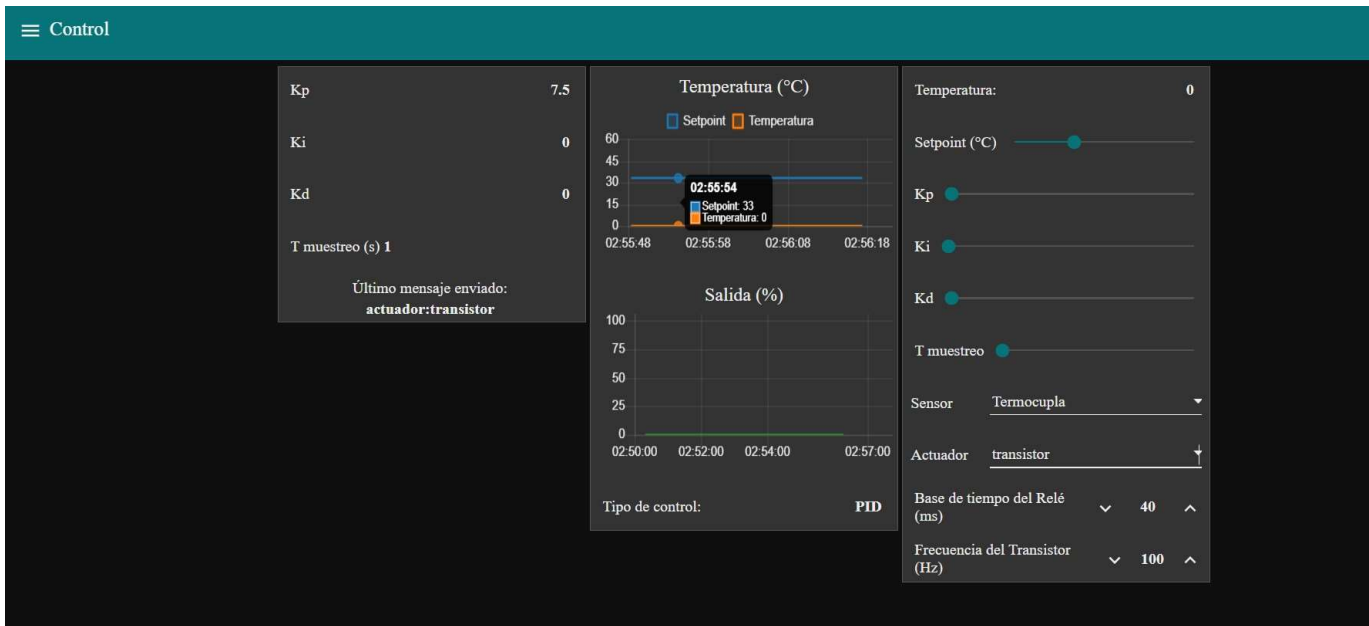


3 Sidebar del proyecto



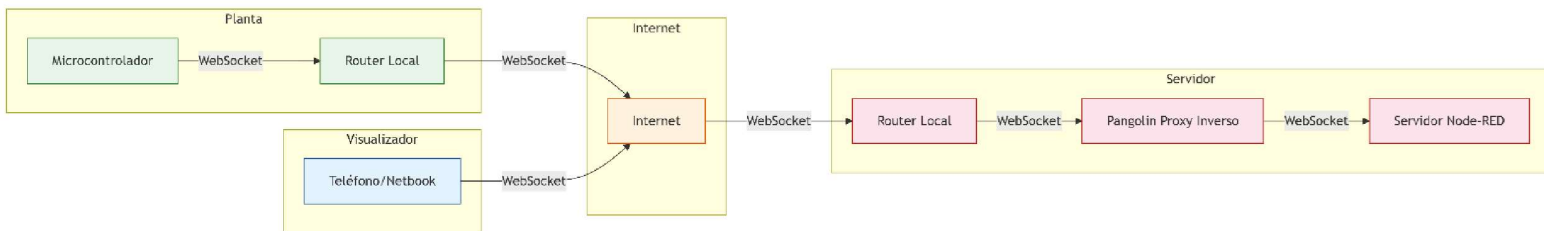


Y con datos se ve así:



Documentación de lo programado:

Si vemos el proyecto como un todo podemos plantear que funciona desde la siguiente manera¹:

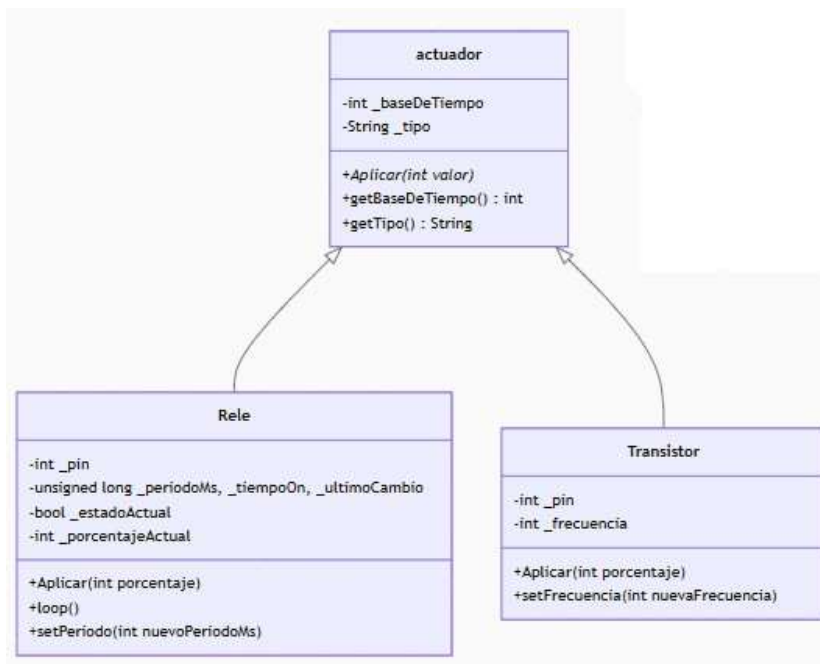
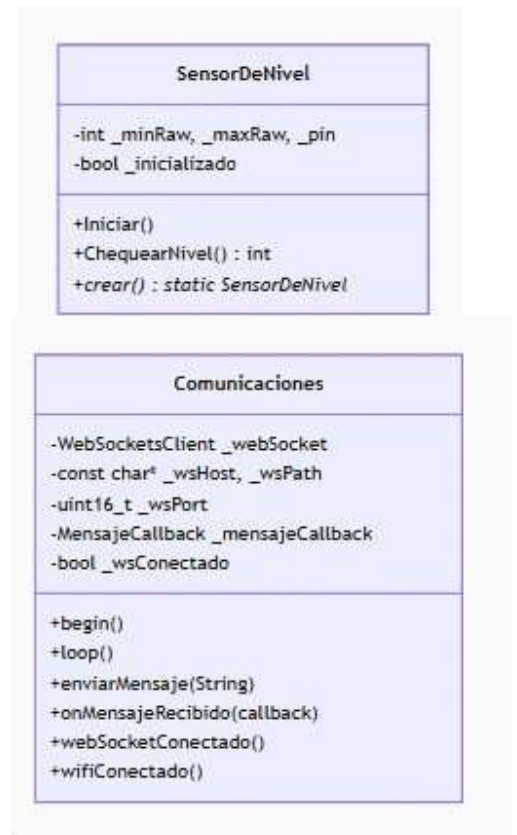
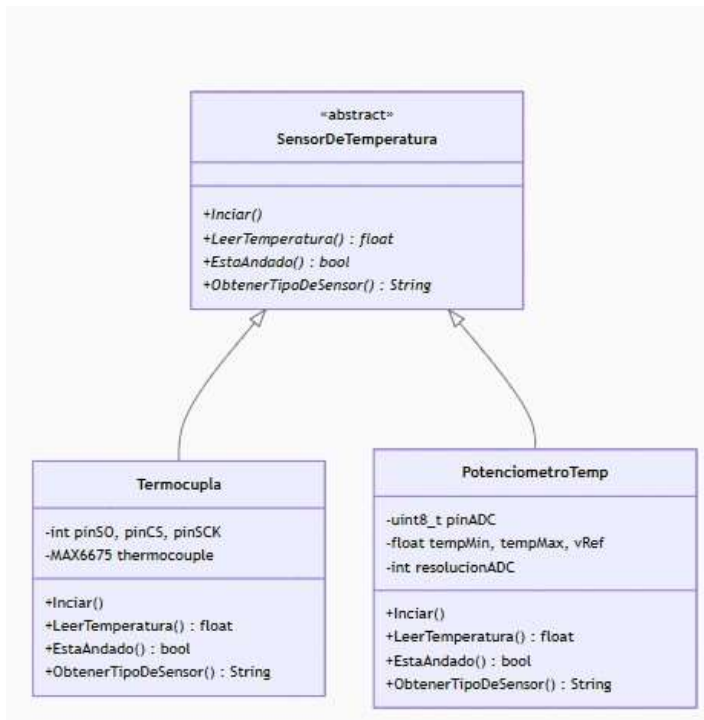


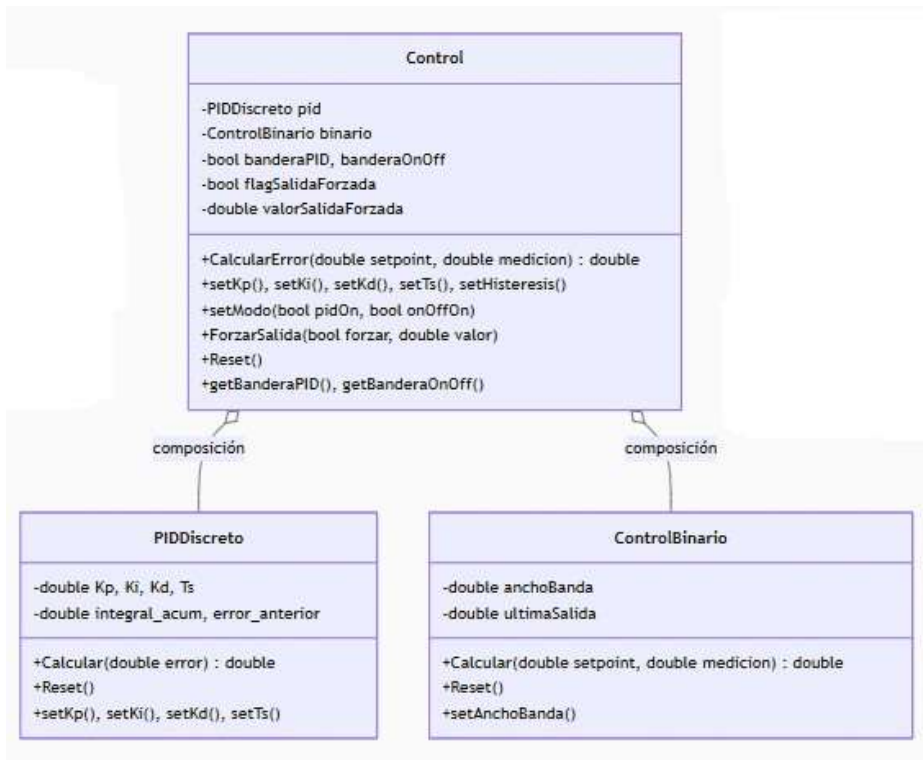
Si “planta”, “visualizador” y “servidor” tienen conexión acceso a internet da igual que el servidor esté en Moscú, la planta en la sede de Concepción de la facultad y la charla se dé en un colegio de Córdoba; aunque en la práctica es tremendamente posible que estén todos en la misma red local.

Clases utilizadas

Intentando, pero no rigurosamente, usar el estándar UML 2.5.1 podemos describir nuestro proyecto con el siguiente diagrama de clase:

¹ De aquí en adelante la mayoría de diagramas y gráficos fueron elaborados con Memraid, Memraid es una herramienta de diagramación y creación de gráficos basada en JavaScript que renderiza definiciones de texto inspiradas en Markdown para crear y modificar diagramas de forma dinámica. Tiene su propio “lenguaje” y tiene renderizado nativo en GitHub, Deepseek, Claude, etc.





Sé intentó que cada clase tuviese un propio Header (.h) y su código fuente (.cpp) a excepción de SensorDeTemperatura (por ser abstracta) y las que componen control (son tan pequeñas que no justifica crear 3 archivos distintos).

Descripción breve de cada clase:

- **SensorDeTemperatura:** Clase abstracta que define el contrato para todos los sensores de temperatura. Obliga a implementar inicialización, lectura, verificación de estado y tipo de sensor. Podríamos decir que es una “interfaz”, pero eso no existe como tal en c++.
- **Termocupla:** Hereda de SensorDeTemperatura. Utiliza el chip MAX6675² para leer una termocupla tipo K mediante SPI. Implementa todos los métodos virtuales.
- **PotenciómetroTemp:** Hereda de SensorDeTemperatura. Simula un sensor de temperatura leyendo un potenciómetro en un pin ADC y mapeando su valor a un rango de temperaturas configurable.
- **Actuador:** Clase base para los actuadores. Almacena el tipo y una base de tiempo (no utilizada directamente). Define el método Aplicar(int) como virtual para que las clases hijas lo sobrescriban.

² Utilizando concretamente la Librería “MAX6675 library” de Adafruit.

- **Rele:** Hereda de actuador. Controla un relé mediante modulación por ancho de pulso (PWM) sobre un período fijo. Implementa un bucle interno (loop()) que actualiza el estado del relé según el porcentaje de encendido.
- **Transistor:** Hereda de actuador. Controla un transistor mediante PWM del ESP32 (función ledc). Permite ajustar la frecuencia de PWM.
- **SensorDeNivel:** Clase independiente que lee un sensor de nivel (por ejemplo, un sensor de agua) mediante un ADC y compara con umbrales para decidir si el nivel es seguro, alto o bajo.
- **PIDDiscreto:** Implementa un controlador PID discreto (posición) con acumulación integral y derivada. Permite actualizar ganancias y tiempo de muestreo.
- **ControlBinario:** Implementa un control ON-OFF con histéresis. Compara la medición con el setpoint y activa/desactiva según la banda muerta.
- **Control:** Clase que compone un PIDDiscreto y un ControlBinario. Ofrece una interfaz unificada para seleccionar el modo de control (PID, ON-OFF o ambos). También permite forzar la salida a un valor fijo (por ejemplo, para seguridad).
- **Comunicaciones:** Gestiona la conexión WiFi (usando WiFiManager con credenciales opcionales en Contraseña.h) y la conexión WebSocket segura (WSS). Envía y recibe mensajes JSON, y permite registrar un callback para procesar mensajes entrantes. Esta usa dos librerías, Websockets de Markus Sattler y WifiManager de Tzapu. Esta última permite que, si la red wifi predeterminada no existe se crea una red wifi llamada controlESP cuya contraseña es control123, esta permite configurar el wifi a otra red que sí esté disponible.

Conceptos de Programación Orientada a Objetos aplicados:

1. Abstracción

SensorDeTemperatura es una clase abstracta que define *qué* debe hacer un sensor, pero no *cómo*. Oculta los detalles de implementación de cada sensor específico. El código principal solo trabaja con punteros a esta clase, sin conocer la implementación concreta.

2. Encapsulamiento

Todas las clases declaran sus atributos como private (o protected cuando es necesario) y exponen métodos públicos para interactuar con ellos. Por ejemplo, Rele oculta su `_periodoMs` y `_tiempoOn`, y solo permite modificarlos mediante `setPeriodo()`.

3. Herencia:

Termocupla y PotenciometroTemp heredan de SensorDeTemperatura, implementando todos sus métodos virtuales. Rele y Transistor tienen como clase padre (heredan de) actuador, sobrescribiendo el método `Aplicar()`.

4. Polimorfismo

El puntero `sensorActivo` de tipo `SensorDeTemperatura*` puede apuntar a un objeto `Termocupla` o `PotenciometroTemp`. Al llamar a `sensorActivo->LeerTemperatura()`, se ejecuta la versión correspondiente según el objeto real (polimorfismo en tiempo de ejecución). Igualmente, `actuadorActivo` puede apuntar a `Rele` o `Transistor`, y `actuadorActivo->Aplicar()` se comporta según el tipo concreto.

5. Composición

La clase `Control` contiene (compone) dos objetos: un `PIDDiscreto` y un `ControlBinario`. No hereda de ellos, sino que los utiliza como partes internas. Esto permite combinar ambos algoritmos de control y delegar en ellos.

6. Uso de punteros y creación dinámica

En el Setup() de nuestro programa principal Sistema.ino (que explicamos más a detalle más adelante) se crean los objetos con new (ej. termocupla = new Termocupla(...)). Esto permite cambiar de sensor o actuador en tiempo de ejecución reasignando los punteros (sensorActivo = termocupla).

7. Métodos virtuales y destructor virtual

SensorDeTemperatura declara un destructor virtual (virtual ~SensorDeTemperatura() = default;) para asegurar que al destruir un objeto a través de un puntero a la clase base, se llame al destructor de la clase derivada.

8. Sobrescritura de métodos (override)

Todas las clases hijas usan la palabra clave override para indicar explícitamente que están redefiniendo un método virtual de la base.

9. Clases estáticas y métodos de fábrica

SensorDeNivel tiene un método estático crear() que retorna un puntero a una nueva instancia. Es un patrón de fábrica simple. Realmente no era necesario, pero queríamos aplicarlo.

Programa principal:

Al ser un proyecto programado en Arduino IDE nuestro programa principal o “main” será Sistema.ino; el

Nótese que si descargas el proyecto desde GitHub este archivo llamado “Contrassenha.h” no existe, esto es porque como dijimos contiene las nuestra red y su contraseña algo que podríamos querer que no sea público por si, por ejemplo, el vecino que pone la música alta encontrase este proyecto. Por eso lo colocamos en el listado de .gitignore. En caso de querer fabricarlo, debe tener la siguiente estructura:

```
// Contrassenha.h - Archivo de credenciales WiFi
#ifndef CONTRASENHA_H
#define CONTRASENHA_H
#define RED "MiRed" // Cambia por el SSID de tu red
#define contrassenha "MiContrassenha" // Cambia por tu contraseña
#endif
```

Posterior a las inclusiones se declaran pines, variables globales, objetos globales y prototipos de las funciones:

```
// ===== DEFINICIÓN DE PINES (ESP32) =====
#define PIN_THERMO_SO 19
#define PIN_THERMO_CS 23
#define PIN_THERMO_SCK 5
#define PIN_POTENCIOMETRO 34
#define PIN_RELE 13
#define PIN_TRANSISTOR 12
#define PIN_NIVEL 39

#define NIVEL_MIN_RAW 400 // Marcamos un mínimo de agua en el sensor,
| | | | | | | | | | // por pruebas se determinó que cuando está con gotas marca ~315
#define NIVEL_MAX_RAW 3300 // 3,3 V
```

```
// ===== PARÁMETROS POR DEFECTO =====  
float SETPOINT_DEFAULT = 25.0; // 25 grados  
float KP_DEFAULT = 7.5;  
float KI_DEFAULT = 0.0;  
float KD_DEFAULT = 0.0;  
double tiempoMuestreo = 1.0; // segundos  
int BASE_TIEMPO_RELE_DEFAULT = 100; // ms (múltiplo de 20 para SSR)  
int FRECUENCIA_TRANSISTOR_DEFAULT = 1000; // Hz  
float HISTERESIS_DEFAULT = 2.0; // grados para control ON-OFF
```

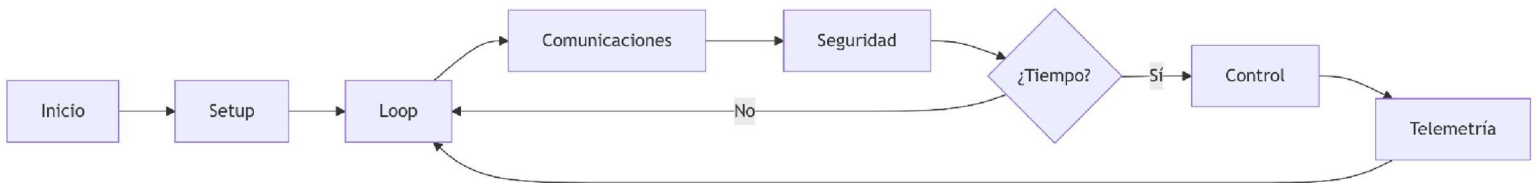
```
// ===== PROTOTIPOS =====  
void procesarComando(String cmd);  
void actualizarActuadorInactivo();  
void verificarNivelYSeguridad();  
void loopControl();  
void enviarTelemetria();  
void alRecibirMensaje(String mensaje);  
void recrearControl(); // ya ☐ se usa para cambiar parámetros, solo para inicial
```

```
// ===== OBJETOS GLOBALES =====  
Comunicaciones comunica;  
// De acá en adelante los declaramos como punteros, ya que luego se crean con new  
// cuando sea conveniente...  
Termocupla* termocupla = nullptr;  
PotenciometroTemp* potenciometro = nullptr;  
SensorDeTemperatura* sensorActivo = nullptr;  
Rele* rele = nullptr;  
Transistor* transistor = nullptr;  
actuador* actuadorActivo = nullptr;  
Control* control = nullptr;  
SensorDeNivel* sensorNivel = nullptr;  
//// ===== VARIABLES DE CONTROL =====  
bool nivelSeguro = false;  
unsigned long tiempoUltimoControl = 0;  
float setpoint = SETPOINT_DEFAULT;  
float ultimaTemperatura = 0.0;  
float ultimaSalida = 0.0;  
String sensorActualStr = "termocupla"; // Por defecto  
String actuadorActualStr = "rele"; // Por defecto
```

Una vez declarado todo, sobretodo con las cosas por defecto; se da inicio al flujo del programa principal.

Flujo del programa principal:

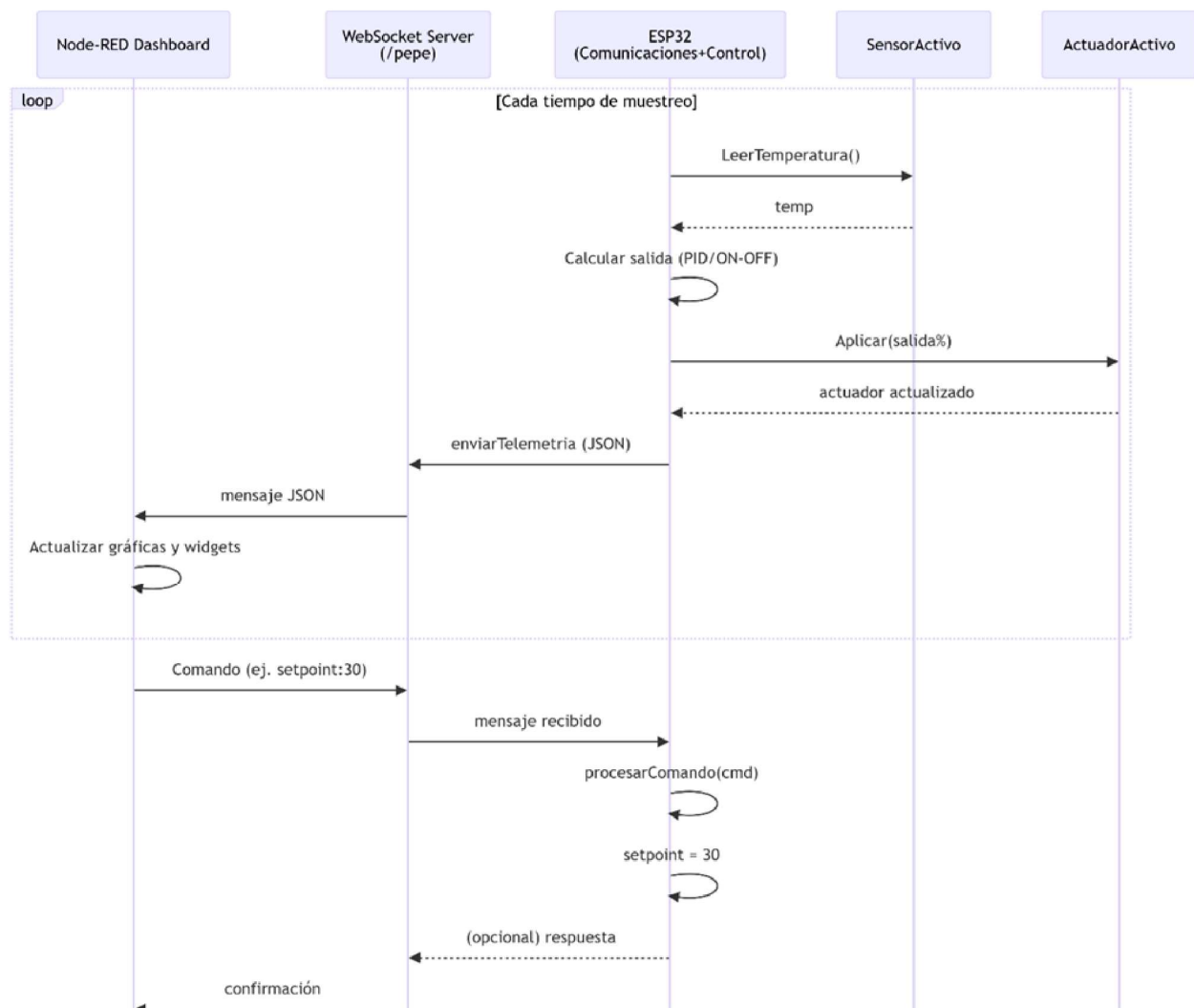
Como cualquier proyecto en Arduino en Setup se inicializa y configura todo (esta parte de ejecuta solo una



vez); y en Loop se ejecuta lo que queremos que haga en si el microcontrolador; esta parte del programa se ejecuta, como indica su nombre, en un ciclo indefinido.

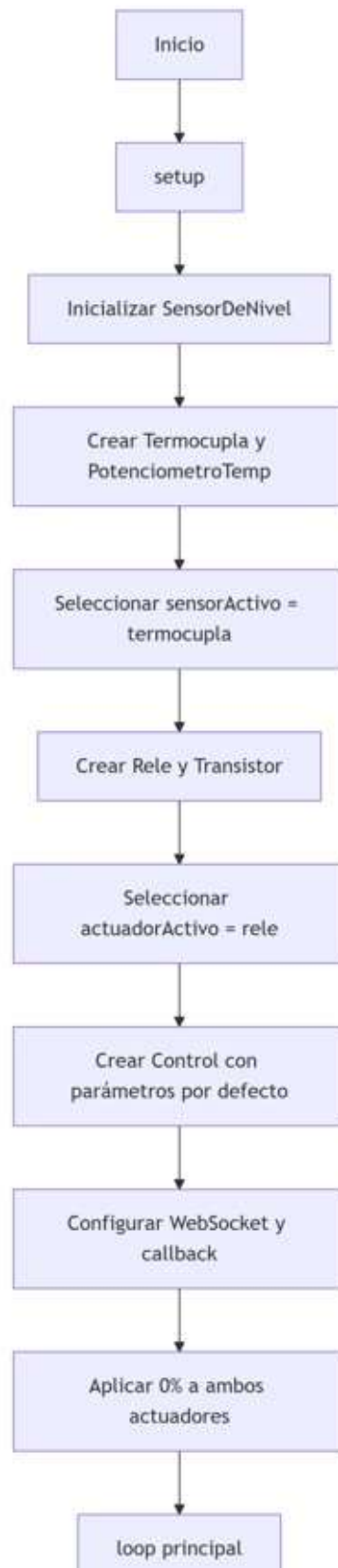
Podríamos resumirlo en el siguiente diagrama de flujo:

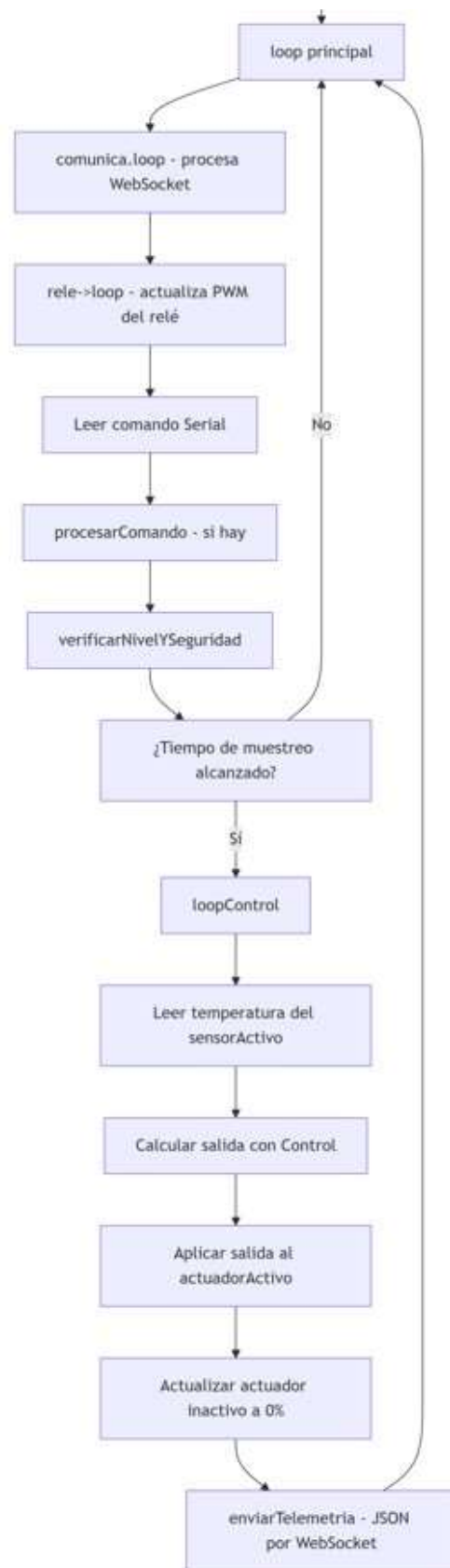
El programa inicia, se configura (Setup), va al Loop, revisa si tiene “mensajes pendientes” (vía comunicaciones por websockets o por puerto Serial), hace un chequeo de la seguridad del nivel de agua, si se cumplió el tiempo entre muestreos (y acciones de control) se ejecuta el control (PID o ONOFF) y se envía la telemetría, volviendo al Loop Principal. El programa es en modo Polling y particularmente lento adrede, para reforzar el carácter digital y discreto de este control (fines pedagógicos). Es muy sencillo entender que es lo que el programa debe hacer si se observa el diagrama de secuencia entre el microcontrolador y el servidor:



Práctica educativa territorial

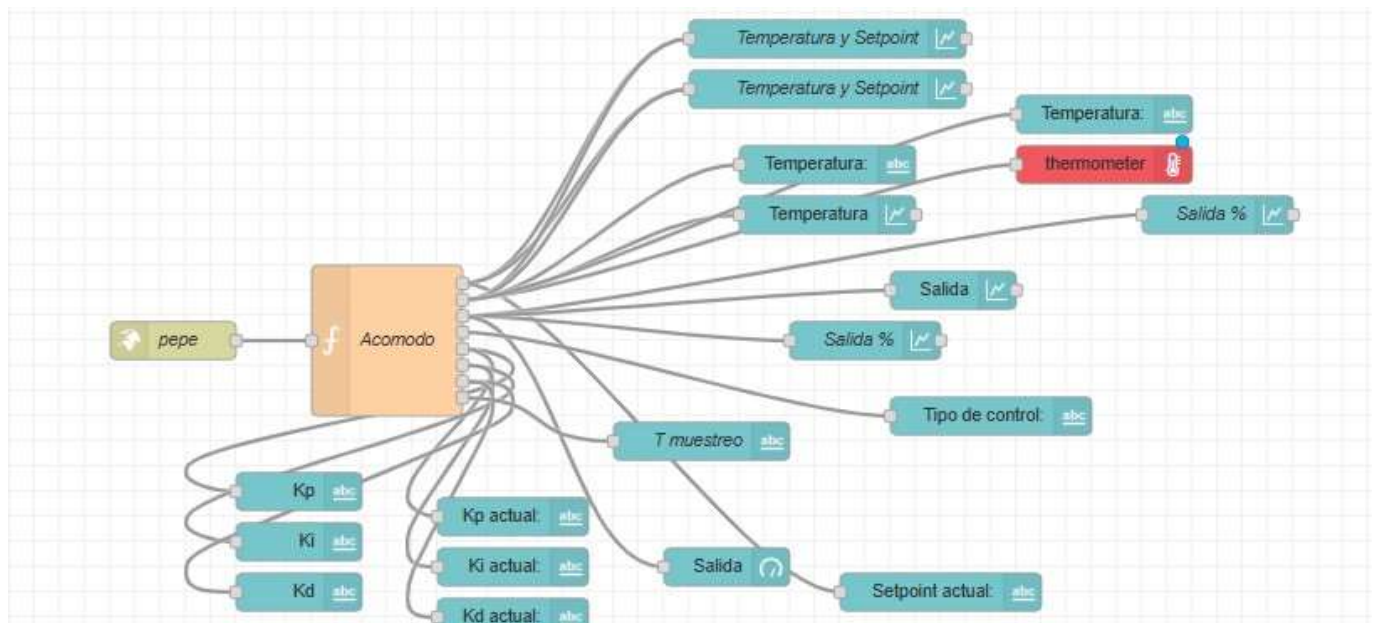
Una vez revisado esto podemos mostrar a más detalle el flujo del programa principal:





Servidor Web/programación Web:

Al ser Node-Red un software de Programación visual el mismo “código” es en sí mismo bastante autoexplicativo. Lo podríamos resumir en las dos cosas que el servidor debe hacer: Visualización de datos y envío de comandos:

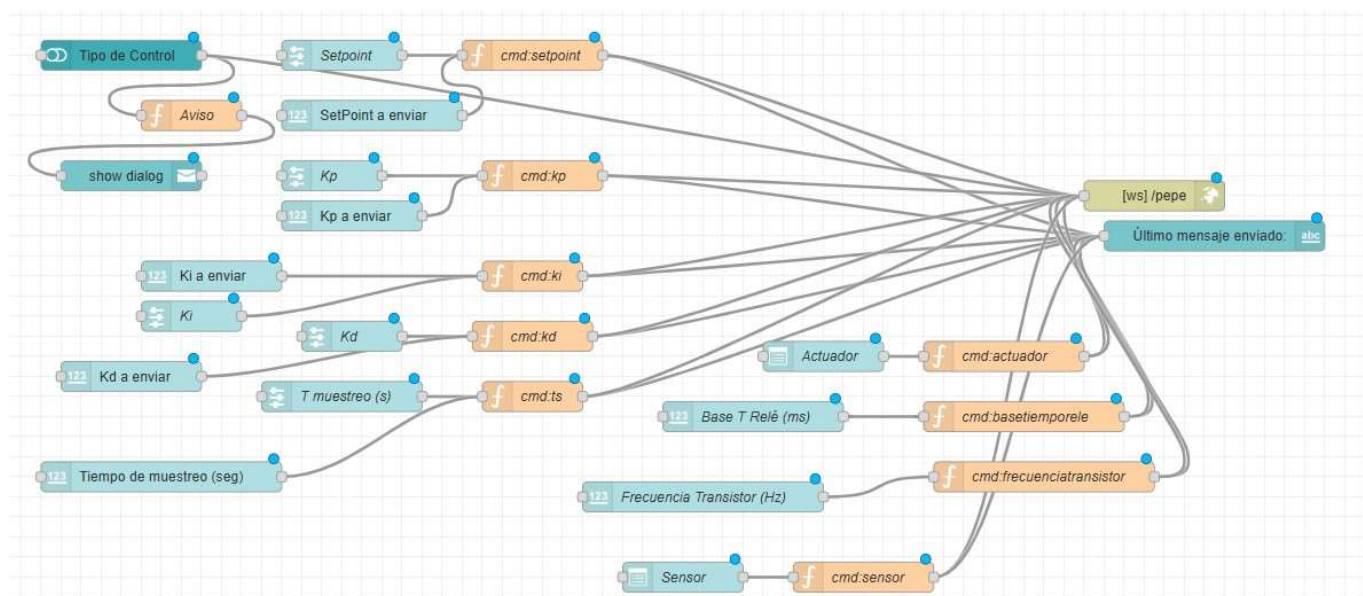


A través de nuestro Websocket, que para fines pedagógicos y humorísticos llamamos pepe, ingresa toda la información que queremos mostrar en sus distintas instancias y presentaciones. En una función llamada Acomodo lo deserializamos y dividimos, con todas las excepciones del caso:

Práctica educativa territorial

```
1 let jsonStr = msg.payload;
2 let telem;
3 try {
4   telem = JSON.parse(jsonStr);
5 } catch(e) {
6   return null; // ignorar líneas que no sean JSON
7 }
8
9 // Preparar array de 8 salidas (todas null por defecto)
10 let outputs = [null, null, null, null, null, null, null, null];
11
12 // Setpoint (salida 0)
13 if (telem.setpoint !== undefined) {
14   outputs[0] = { topic: "Setpoint", payload: telem.setpoint };
15 }
16 // Temperatura actual (salida 1)
17 if (telem.temperatura !== undefined) {
18   outputs[1] = { topic: "Temperatura", payload: telem.temperatura };
19 }
20 // Porcentaje de salida (salida 2)
21 if (telem.salida !== undefined) {
22   outputs[2] = { payload: telem.salida };
23 }
24 // Modo de control (salida 3)
25 if (telem.modo !== undefined) {
26   outputs[3] = { payload: telem.modo };
27 }
28
29 // Parámetros PID si existen (salidas 4 a 7)
30 if (telem.modo === "PID") {
31   if (telem.kp !== undefined) outputs[4] = { payload: telem.kp };
32   else outputs[4] = { payload: "N/A" };
33   if (telem.ki !== undefined) outputs[5] = { payload: telem.ki };
34   else outputs[5] = { payload: "N/A" };
35   if (telem.kd !== undefined) outputs[6] = { payload: telem.kd };
36   else outputs[6] = { payload: "N/A" };
37 } else {
38   // Modo ONOFF o MIXTO: no hay PID, mostramos "N/A"
39   outputs[4] = { payload: "N/A" };
40   outputs[5] = { payload: "N/A" };
41   outputs[6] = { payload: "N/A" };
42 }
43 // Tiempo de muestreo (salida 7)
44 if (telem.ts !== undefined) {
45   outputs[7] = { payload: telem.ts };
46 }
47 return outputs;
```

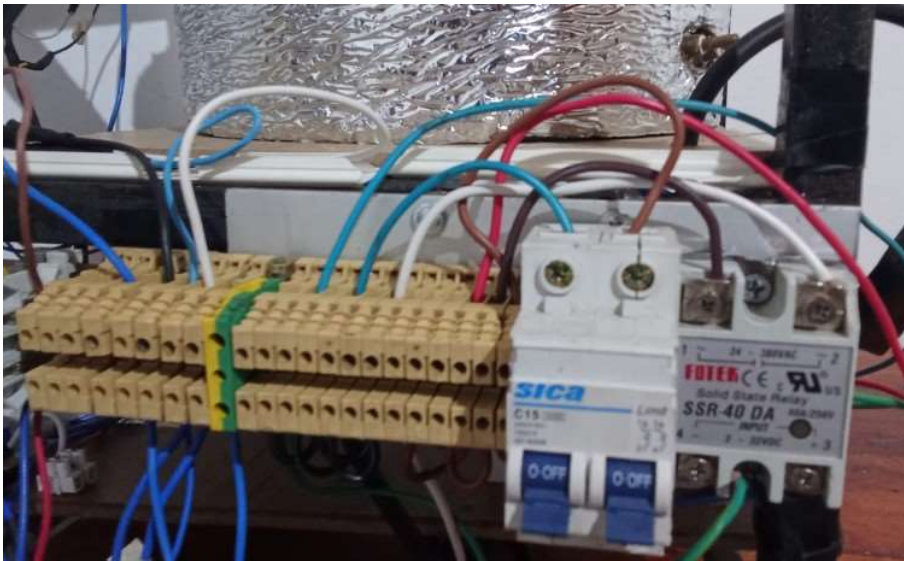
Por su parte la sección de envío de comandos al microcontrolador es simple, controles (botones, slides, etc) y funciones que les dan formato para luego enviarlos al mismo WebSockets.



Podríamos pensarlo como un programa con dirección por eventos, donde podemos dividir los eventos en dos grandes grupos: la telemetría (constante) y lo que hace el usuario (interactuar con los elementos de GUI).

Construcción física:

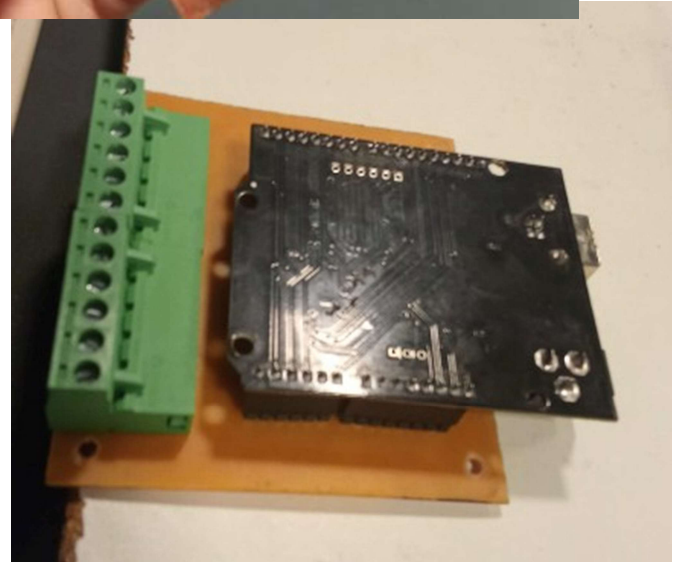
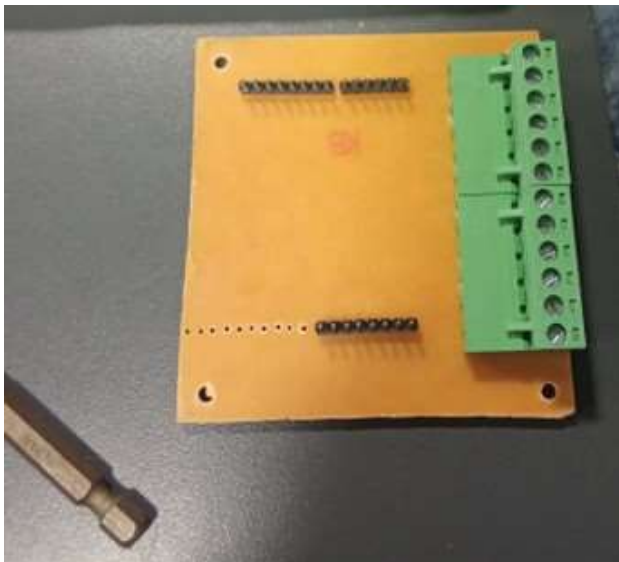
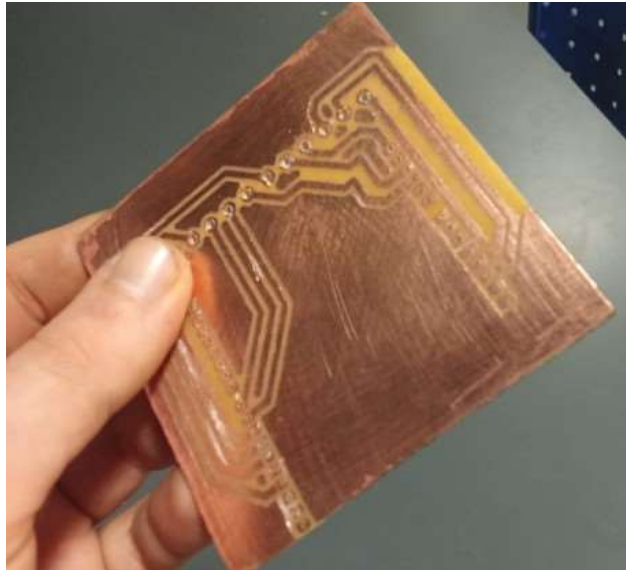
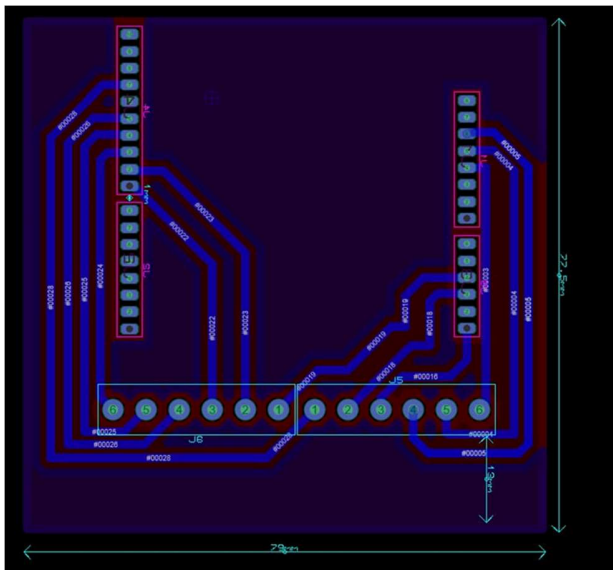
Se notará que muchos elementos se encuentran sobredimensionados, esto se debe a que fueron quedando de proyectos anteriores y es innecesario gastar de nuevo dinero en ellos, además los componentes “grandes” como el relé SSR o el transistor con un disipador grande es bastante pedagógico para cuando se explique el sistema.



Para el encendido y apagado del relé utilizamos un circuito con un transistor bjt (2n222), ya que si bien el relé es de 3 v como el controlador, este no otorga la corriente suficiente:



Por su parte, en la práctica, los cables Dupont (o “tipo Arduino”) presentaron múltiples errores por falsos contactos; por lo que se añadió una “placa madre”, con borneras para que las mismas estuvieran bien sujetas:



Detalles Adicionales:

- Se usó Git para el control de versiones, con un desarrollo en tronco.
- Se adjuntó un archivo por lotes (.bat) en la sección Web para una ejecución más sencilla del servidor, sin tener que abrir nada en particular:



```
@echo off
node-red "%~dp0flow.json"
pause
```

- En caso de que salte un error al ejecutarlo (que no debería, pero puede ser si el sistema operativo del servidor no es Windows o si se realizó una instalación de node red sin ejecutable), hay que seguir los siguientes pasos:
 - 1) Asegurarse que npm, Node.js y Node red se encuentran instalados y funcionando, caso contrario instalarlos.
 - 2) Instalar el nodo “dashboard” como se indica aquí:

node-red-dashboard 3.6.6

A set of dashboard nodes for Node-RED

```
npm install node-red-dashboard
```

Es decir, en el cmd o Shell (según corresponda) se ejecuta “npm install node-red-dashboard”.

- El sensor de la termocupla es bastante sensible, precisa de permanecer estática para que las mediciones no sean erradas

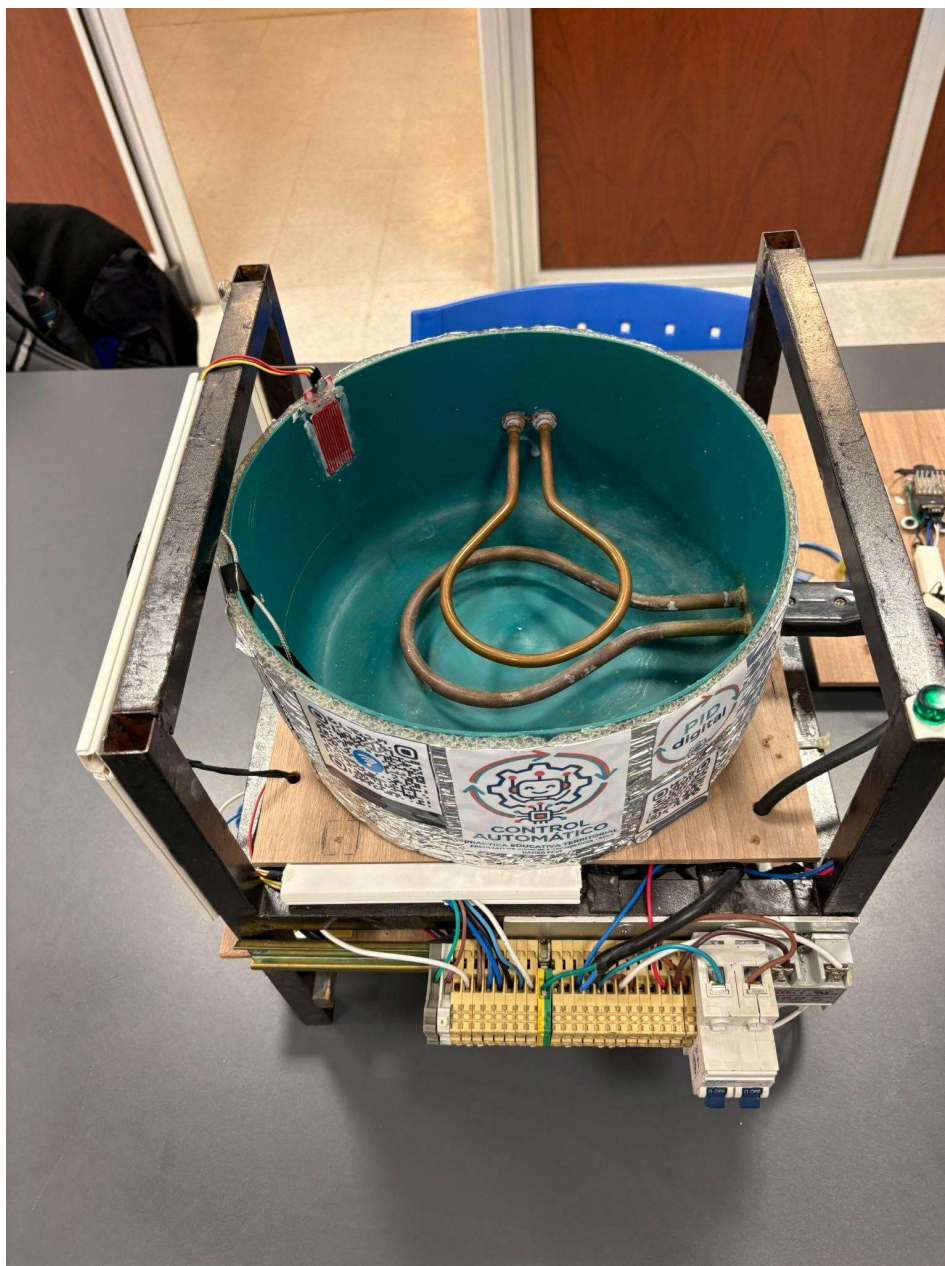
Bibliografía:

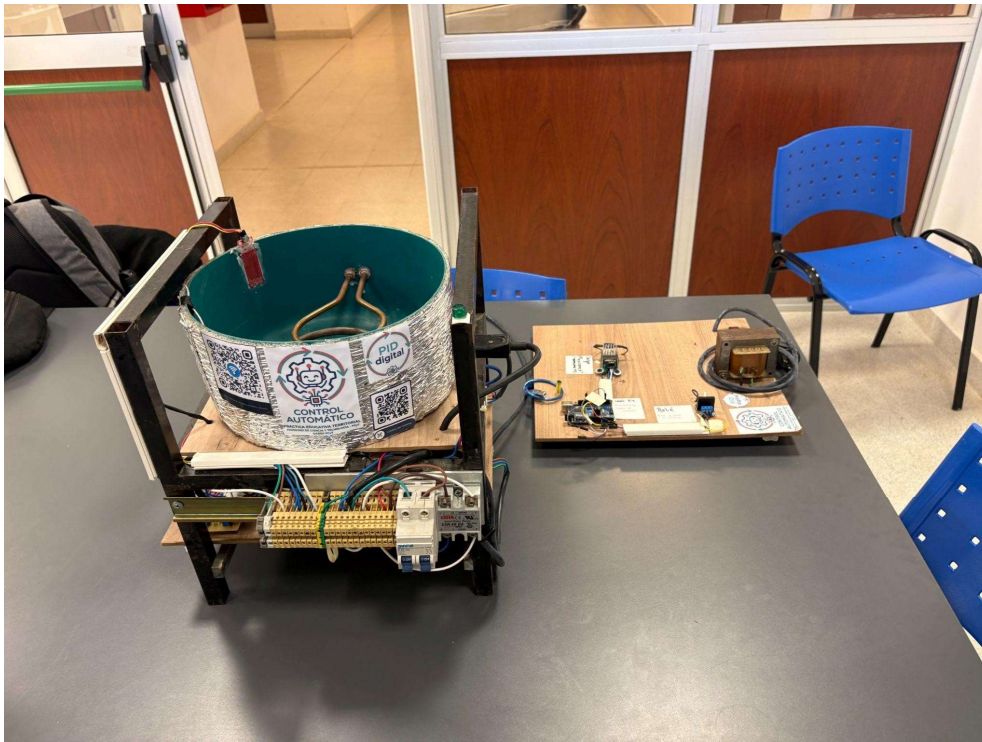
- Garcia, R. (2025, 24 de octubre). *Los 4 pilares de la programación orientada a objetos en Java*. Medium. <https://medium.com/@rodrygocgarcia/los-4-pilares-de-la-programaci%C3%B3n-orientada-a-objetos-en-java-5291dfc258bd>
- Pangolin. (n.d.). Introduction to Pangolin. Pangolin Docs. Recuperado el 20 de junio de 2026, de <https://docs.pangolin.net/>
- Node-RED Documentation. (s.f.). Node-RED. <https://nodered.org/docs/>
- Geekworm. (s.f.). *ESP8266-32 Guide-CN* [Archivo PDF]. Recuperado de <https://wiki.geekworm.com/images/e/ee/ESP8266-32-Guide-CN.pdf>
- Shenzhen Doit Technology Co., Ltd. (s.f.). *ESP8266-32 Guide-CN* [Archivo PDF]. Publicado en Geekworm Wiki. <https://wiki.geekworm.com/images/e/ee/ESP8266-32-Guide-CN.pdf>
- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1>
- Sveidqvist, K. (Ed.). (s.f.). *Mermaid: Diagramming and charting tool* [Software]. <https://mermaid.js.org/>
- Node-RED contributors. (s.f.). *node-red-dashboard* [Software]. Node-RED Flow Library. <https://flows.nodered.org/node/node-red-dashboard>

Práctica educativa territorial

Vistas de PID terminado







Presupuesto estimado:

Enchufe macho: \$4.500

2 toma corrientes: \$4.000 cada uno

Fuente de alimentación 220-19: \$39.000

Cables de Arduino macho y cables de Arduino hembra: \$4.000 cada uno

Borneras: \$12.000

ESPduino: \$14.000

Potenciómetro 10K: \$4.000

Sensor resistivo de nivel: \$2.600

2 resistencias tipo herradura (de termotanque): \$6.000 cada una

2 Conectores hembra para resistencia en herradura: \$2.000 cada uno

SSR-40 DA: \$20.000

Práctica educativa territorial

Módulo MAX 6675 + termocupla tipo k: \$10.000

Módulo Mosfet autoacoplado:\$10.000

Cable USB tipo A (tipo Arduino): \$5.000

Riel Dim: \$4.300

Cargador de teléfono 5V: \$4.000

Balde de limpieza: \$5.000

Llave termica: \$22.000

Cable canal de 2mm: \$4.000

Total: \$182.400 (121 USD)